OblivRec: Private Serving and Delivery

Mid-Term Technical Report, CS670

Mainak Sarkar (230619) Shrasti Dwivedi (230982) Aditya Anand (230065)
Shravan Agrawal (230984)

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur
12 September 2026

Abstract

This report records the first implementation slice of OblivRec, a recommender that composes the three-party private training core of Nudge with the private delivery layer of Pirsona. The slice covers private serving and delivery against a model trained in the clear, which is stage S1 of the staging fixed in the requirements. We report a distributed point function written from the Boyle–Gilboa–Ishai construction, a DPF-based private information retrieval read layer over the MovieLens catalogue, a cleartext power-iteration factorisation used as the quality oracle, and an experiment quantifying what an observer learns from watching which items a user fetches. Private training itself is out of scope here and is scheduled for the following phase.

Contents

1 Scope of this slice

The full system has two halves. Private *training* learns a factorisation of a rating matrix that no server may see, and private *delivery* fetches the recommended item without revealing which item it was. The requirements document fixes the build order as S1, then S2, then S3: serving and delivery first, then training, then the composition. S1 is first because it is lower risk, it demonstrates something end to end early, and the distributed point function it needs is a prerequisite for the non-linear gates that private training will need later.

This report covers S1 only. The model is trained *in the clear* by the Python oracle of Section 2, and everything downstream of that model is private. Private training is the subject of the next phase and no claim is made about it here.

1.1 What is deliberately absent

Three components specified in the design draft are not built in this slice, and each was cut for a stated reason rather than for lack of time.

A distributed comparison function is not implemented. Its only consumers are the truncation and normalisation protocols, both of which belong to private training. The design draft asserts that a distributed point function and a distributed comparison function are the same primitive. They are not, and the reference implementation of Nudge keeps them in separate modules, which corroborates the distinction.

Oblivious top-k selection is not implemented. The user reconstructs the score vector and selects the top k locally, so no server learns the selection regardless of whether an oblivious selector exists. The apparatus would protect something that is already protected.

Network emulation is not used. The development machine has neither Docker nor `tc netem`, so every measurement in this report is taken on the `local` profile and is labelled as such. Wide-area numbers are deferred rather than estimated.

Networking itself is not implemented. The three servers in this slice are three sets of shares, held and combined in a single process exactly as the protocol specifies; there is no socket between them. The consequence is worth stating directly rather than leaving to be inferred. Every message in the design has an input-independent size, and every expanded Dpf coefficient is a pseudorandom share rather than a mostly-zero vector, so an observer on the wire would have nothing to distinguish — but that is an argument from the construction, not a demonstration. The channel-boundary transcript and the distinguisher experiment that would demonstrate it have not been run. This is the largest open gap in the security story of this slice, and it is named here, in `docs/threat-model.md`, and in `PHASES.md` rather than being allowed to pass silently.

2 The cleartext oracle, and how many power iterations are needed

Every private component in this project is checked against a cleartext twin. The factorisation oracle is therefore the first thing built, before any cryptography, and it doubles as baseline B1, the speed-of-light reference against which recommendation quality is measured.

2.1 Construction

The oracle implements ApproxFactor as Nudge states it, rather than calling a singular value decomposition. This is deliberate. The private implementation in the next phase must reproduce this procedure step by step under secret sharing, so the reference has to match the *structure* of

the protocol and not merely its output. Concretely, for each of d components a random vector is repeatedly multiplied by $\mathbf{U}^\top\mathbf{U}$, made orthogonal to the components already found, and normalised. Each converged component is then revealed in the clear and becomes a row of $\mathbf{B}$.

Revealing each row before computing the next is what keeps the orthogonalisation cheap, because it is then a Gram–Schmidt step against public vectors. It is also the reason the item embedding matrix is public to every server, which is the premise of the leakage analysis in this project.

A singular value decomposition is still computed, but as a *cross-check* rather than as the implementation. Power iteration on $\mathbf{U}^\top\mathbf{U}$ converges to the top-d right singular subspace, so the largest principal angle between the two subspaces should approach zero. That check costs two lines and it caught a genuine question on the first day.

2.2 Quality is flat in the iteration count

At the default of $\ell = 10$ inner rounds the subspace angle is 9.66×10^{-1} radians, which is nowhere near zero. Sweeping ℓ resolves what that means.

Table 1: MovieLens-100K, split `u1`, $d = 16$. The subspace angle falls by five orders of magnitude while recommendation quality does not move. Raw data in `bench/results/oracle_ell_sweep.jsonl`.

ℓ	subspace angle (rad)	nDCG@20	Recall@20
10	9.659×10^{-1}	0.4536	0.4693
25	4.170×10^{-1}	0.4545	0.4677
50	1.834×10^{-1}	0.4525	0.4650
100	6.966×10^{-2}	0.4523	0.4662
200	3.604×10^{-3}	0.4526	0.4664
400	6.652×10^{-6}	0.4526	0.4664

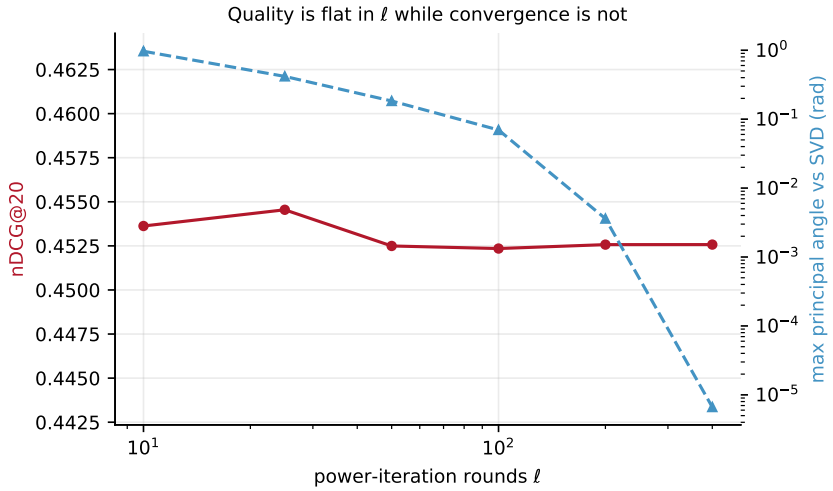

Figure 1: The same data as Table 1, plotted. Note the axes: nDCG@20 spans a range of 0.002 while the subspace angle spans five orders of magnitude. Since ℓ multiplies the only interactive cost in private training, the flat curve is what makes a fortyfold communication saving free. Regenerated by `mingw32-make figures`.

Two conclusions follow, and they are separate.

The first is that the implementation is correct. The angle falls monotonically to 6.7×10^{-6} radians, so the procedure does converge to the right subspace. The large angle at $\ell = 10$ is slow convergence rather than a defect. The spectrum explains the rate: the ratio of the sixteenth to the seventeenth squared singular value is 1.032, a gap of three percent, and power iteration converges as a power of that ratio. The first component is easy, with a ratio of 6.44. The sixteenth is not.

This distinction was nearly lost. A failing cross-check invites the reaction of removing the cross-check, which would have converted a correct implementation into a silently wrong one.

The second conclusion is the useful one. Across a fortyfold increase in ℓ, nDCG@20 stays between 0.4523 and 0.4545, a spread of about half a percent with no monotone trend. Ranking does not require the singular vectors, only a subspace good enough that the ordering of scores is stable.

2.3 Why this matters for private training

Under two-out-of-three replicated sharing the matrix–vector products are non-interactive and therefore free. Truncation and normalisation are the entire communication cost of training, and they run once per inner iteration, so communication is linear in ℓ.

The measurement therefore says that private training can run at $\ell = 10$ and the fortyfold saving in communication costs nothing in recommendation quality. That is a design decision resting on evidence rather than on the default value in the design draft, and it also discharges an instruction the draft had already given and which had not been acted on, namely to measure the residual against the cleartext oracle and report iterations against quality.

The caveats are stated plainly. This is one dataset, one split, one seed and one embedding dimension. The flatness may not survive at MovieLens-1M or at $d = 32$, and it will be re-measured before the final report relies on it. The metric uses binary relevance at a rating of four or above, so these numbers are a within-project baseline and are not directly comparable to the figures Nudge reports on Netflix data under its own convention.

3 The distributed point function

A point function $f_{\alpha,\beta}$ takes the value β at $x = \alpha$ and zero everywhere else. A $(2, 2)$-distributed point function splits it into two keys, each of which individually hides both α and β, such that the two evaluations recombine to $f_{\alpha,\beta}(x)$. The construction is that of Boyle, Gilboa and Ishai, and it is written by us rather than imported, because it is the component a viva will probe hardest and because the same implementation serves both halves of this project: the private read layer in Section 4, and the function secret sharing gates that private training will need in the next phase.

3.1 Why the keys are small

The construction is a binary tree of pseudorandom seeds. Each key holds one initial seed and one correction word per level, so key size is *logarithmic* in the domain while a naive one-hot query is linear. Table 2 gives measured sizes from the serialiser rather than from the closed form, since the closed form is what the design draft got wrong.

The design draft claims roughly 260 bytes at $n = 4096$ against 32 KB naive. The 32 KB figure is right. The 260-byte figure is not: the measured value is **245 bytes**, and the discrepancy comes from the draft's formula omitting both the initial seed and the self-describing header, while simultaneously evaluating at 16 domain bits rather than the 12 that $n = 4096$ implies. The number in this report is measured by a test that prints it, which is why the error surfaced.

Table 2: Measured DPF key size against the naive alternative. The packed format is $1+4+16+18\,\mathrm{db}+\lceil b/8\rceil$ bytes, and `SizeBytes()` is asserted equal to the actual serialised length. The naive column and the compression ratio are both taken at $b = 64$, so the ratio understates the $b = 128$ case, where a one-hot query would itself double. These figures are printed by `tests/test_dpf_serialize.cpp` and copied here, not the other way round.

domain bits	n	key, $b = 64$	key, $b = 128$	naive, $b = 64$	compression
10	1,024	209 B	217 B	8,192 B	39×
11	2,048	227 B	235 B	16,384 B	72×
12	4,096	245 B	253 B	32,768 B	134×
16	65,536	317 B	325 B	524,288 B	1654×

3.2 The sign convention

Textbook presentations of this construction define $\mathrm{Eval}_b = (-1)^b\big(\mathrm{Convert}(s) + t \cdot \mathrm{CW}_{\mathrm{last}}\big)$, which yields the *sum* convention $\mathrm{Eval}_0(x) + \mathrm{Eval}_1(x) = f(x)$. The specification for this project instead states the invariant as a *difference*. We therefore omit the $(-1)^b$ factor and return $\mathrm{Convert}(s) + t \cdot \mathrm{CW}_{\mathrm{last}}$ directly, giving

$$\mathrm{Eval}(k_0, x) - \mathrm{Eval}(k_1, x) = \begin{cases} \beta & x = \alpha \\ 0 & \text{otherwise.} \end{cases}$$

The two conventions are algebraically identical and differ only in where the negation is placed. The reason to record the choice explicitly is that getting it backwards makes every test fail in a way that resembles a tree traversal bug rather than a sign error, which is an expensive place to spend a day.

3.3 Where the key material comes from

The initial seeds are the entire secret. Every correction word in a key is public-looking by construction, and the security argument rests on an adversary being unable to guess or reconstruct the two initial seeds.

The first version of this code seeded a Mersenne Twister and used its output as seed material. That was a genuine break rather than an infelicity. A Mersenne Twister is fully reconstructible from 624 consecutive outputs, so an adversary who observed enough key material could rebuild the tree and recover α, which is precisely the value the construction exists to hide. The seeding also supplied roughly 64 bits of entropy against a claimed security parameter of $\lambda = 128$.

Key material is now drawn from the operating system generator through the Windows CNG interface. The failure policy is to abort rather than fall back to a weaker source, because a silent fallback would leave a broken guarantee indistinguishable from a working one. The accompanying tests check the properties that actually fail in practice: that successive draws differ across two thousand samples, that output is not stuck at a constant, that the bit balance and byte spread are sane, and that odd request lengths neither overrun the caller's buffer nor leave part of it unwritten. None of that establishes cryptographic quality, which no test suite can, and the report does not claim otherwise.

3.4 Testing: the structural invariant, checked at every node

A black-box test reports only that a leaf is wrong. The characteristic failure of this construction is a mistaken control-bit update, which stays invisible until enough levels have accumulated for it to

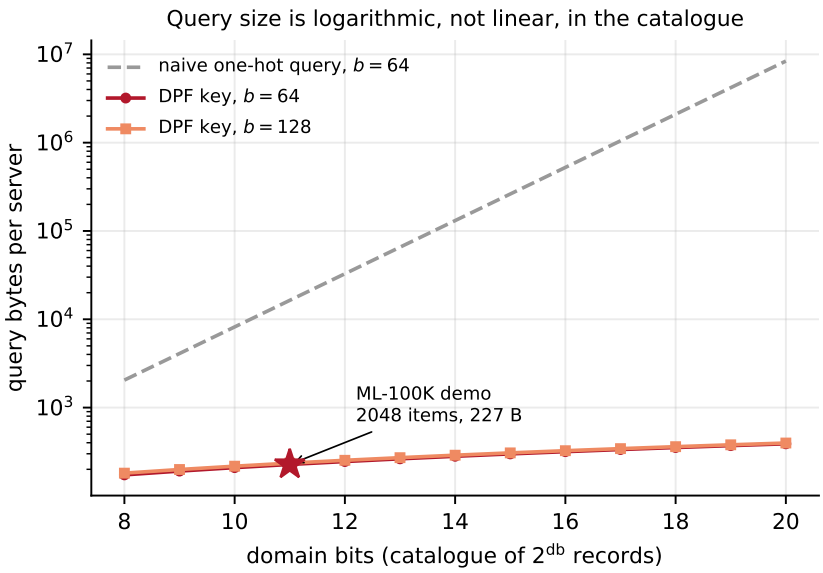

Figure 2: The same closed form as Table 2, extended. The gap widens without bound: the naive query doubles with every domain bit while the key grows by 18 bytes. The starred point is the operating point the demo runs at. Regenerated by `mingw32-make figures`; the closed form plotted here is the one `tests/test_dpf_serialize.cpp` asserts equal to the actual serialised length, so the figure and the test cannot drift apart.

change an output, and therefore presents as "correct at two domain bits, wrong at eight". Bisecting that from leaf values is slow.

The test suite therefore walks *both* parties' trees in lockstep and asserts the underlying structural invariant at every internal node. Writing (s_b, t_b) for party b's state:

On the path to α	$s_0 \neq s_1$ and $t_0 \oplus t_1 = 1$
Off the path	$s_0 = s_1$ and $t_0 \oplus t_1 = 0$

Off the path the two parties hold identical state, which is precisely why their contributions cancel. On the path they differ and their control bits disagree, which is what lets the final correction word inject β at exactly one leaf. Every correctness property of the scheme follows from these two lines, so a violation names the exact level and node at which the construction first went wrong.

This checker was written *before* the generator was trusted, and both were exercised together on the first run.

3.5 Correctness results

The structural invariant holds at every node for every α in every domain up to eight bits, and at sampled α up to eleven bits.

Exhaustive correctness is two claims, not one. Treating it as one is what made the requirement look infeasible. The specification asks for exhaustive correctness up to sixteen domain bits, and it states the invariant over *whole-domain* evaluation. Those are different costs. Checking every α against every x costs 4^D leaf comparisons either way, but reaching them through one whole-domain

pass per α costs two tree walks, whereas reaching them through single-point evaluation costs D tree walks per index. At sixteen bits that is a factor of eight. So the whole-domain formulation carries the headline claim, which is also the formulation the specification itself uses, and single-point evaluation is bridged to it separately.

Claim one, the difference invariant. For every α and every x, $\mathrm{EvalFull}(k_0)[x] - \mathrm{EvalFull}(k_1)[x]$ equals β at $x = \alpha$ and zero elsewhere. The default test target establishes this for every domain up to eleven bits on both rings, in under a second. Eleven bits is not an arbitrary stopping point: MovieLens-100K has 1682 items which pad to a 2048-entry domain, so it is the domain the demonstration actually runs.

The full sweep runs from a separate target and has been executed. It covers every α against every x for every domain up to **sixteen bits on the 64-bit ring and fifteen on the 128-bit ring**, which is 5.73 billion and 1.43 billion leaf comparisons respectively, 7.16 billion in total. It passed, at commit `bce4cb1` on 7 September 2026, in **27 minutes**, sustaining about 4.4 million leaf comparisons per second. The 128-bit ring stops one bit lower deliberately, because its comparisons cost roughly twice as much and the marginal bit would have added a further twenty minutes for no new structural coverage.

It is kept out of the default suite for the obvious reason: a suite that takes half an hour stops being run, and then it stops catching anything.

Claim two, agreement. Single-point evaluation is checked index by index against the whole-domain arrays, exhaustively over every α to eight bits on both rings and at chosen α beyond that. Composing the two claims gives exhaustive correctness for single-point evaluation over the same range.

The α values chosen above the exhaustive bound are not purely random. They are the first index, the last index, and the two alternating bit patterns, then a seeded random fill, because most significant bit first indexing errors show up at exactly those points.

One value of β is not a test of β. An earlier version of this sweep used a single fixed β per ring, so billions of leaf comparisons all exercised the final correction word at one value. That word is built with a sign that depends on a control bit, and it is the subtlest line in the generator. The sweep now derives β from the domain size and α through a mixing function, which costs nothing, keeps failures reproducible, and multiplies the number of distinct β values tested by the size of the domain. The explicit edge cases remain: α at the first and last index, $\beta = 0$ where everything must cancel, $\beta = -1$ where the ring wraps, and a separate check that $\beta = 1$ reconstructs exactly, since the read layer depends on that with no tolerance for an error term.

Traversal is implemented once and shared by single-point evaluation, whole-domain evaluation and the test checker, so there is one traversal implementation to be wrong rather than three.

3.6 The wire format, and a bug found by testing it

Whole-domain evaluation and key serialisation were written alongside the header, since the header is a frozen cross-component contract and had to be complete, but they were initially exercised only indirectly. Testing them afterwards found one real defect.

Deserialisation accepted a party byte outside $\{0, 1\}$. This is not cosmetic. Evaluation begins with t set to the party index and the traversal step branches on whether t is non-zero, so a party byte of, say, 7 is truthy and the key would *silently evaluate as party one*. A corrupted or malicious key therefore produced a wrong reconstruction with no error raised anywhere. Since keys arrive over the wire, the fix is to validate on entry rather than to trust the sender. The domain size is now

validated on the same principle, before it is used either to size an allocation or to multiply out an expected length.

The tests that found it are retained as regression tests rather than discarded. They cover round-tripping on both rings, including that a full pair of keys still reconstructs β after both have crossed the wire, and they check that truncated, over-length, empty, and malformed-header inputs are all rejected. Whole-domain and single-point evaluation are asserted to agree at every index, and whole-domain evaluation is required to reject a wrongly sized output buffer rather than overrun it.

Two further checks were run and left in place. The decimal formatter for the 128-bit ring writes thirty-nine digits into a forty-byte buffer, so it has zero margin at the maximum representable value and that value is now pinned by a test. And the ranking metric used for every quality number in this report was cross-checked against an independent implementation over two hundred random instances, agreeing on all of them, together with the boundary cases of a perfect ranking, a worst ranking, a user with no relevant items, and correct exclusion of items already seen in training.

3.7 Hardened verification

The code is clean under a strict warning set including implicit conversion and sign conversion diagnostics. Beyond that, the whole suite runs under a hardened build, available as a build target so that it is repeatable rather than a one-off.

The compiler on this machine ships no sanitizer runtime library, so the usual address and undefined-behaviour sanitizers cannot link. Undefined-behaviour checking is still available in *trap* mode, where a violation compiles to an illegal instruction rather than a call into a diagnostic runtime, and that needs no library. The suite is clean under it.

A clean run under an instrumentation that silently did nothing would look identical to a clean run under one that worked, so both instrumentations were verified against deliberate faults. A program performing an out-of-range shift runs to completion and prints a nonsense value when uninstrumented, and dies on an illegal instruction when instrumented. The bounds check described below behaves the same way.

Checking the standard library covers containers and iterators, but it does nothing for a hand-written type. That mattered here. The local span type is the only unchecked raw-pointer indexing anywhere in this codebase, and it carries the deserialisation path, which parses attacker-controlled bytes. The hardened build was therefore leaving precisely the most exposed path unverified. The span now performs its own bounds check under the same macros, reporting the offending index, the size and the source location before aborting. Under a release build it costs nothing.

The remaining instrumentation gap is a genuine address sanitizer, which would catch heap buffer overflows, use after free and leaks. Its principal failure classes do not arise here, because the code performs no manual heap management at all: there is no occurrence of `new`, `delete`, `malloc` or `free` in the source, and every owning container is a standard vector. That is a weaker statement than having run the tool, and it is recorded as such rather than presented as equivalent.

4 Private serving and private delivery

This section covers the half of the system that is built and running: the user is scored against the catalogue without any server learning the user's embedding, and the resulting records are fetched without either serving party learning which records were fetched. Private *training* is the next phase and is not claimed here.

4.1 The catalogue is fixed-width, and that is a security property

A private read must return the same number of bytes whatever it reads, or the response length itself leaks the record. MovieLens-100K's `u.item` is variable-length text, so it is packed into a fixed record of $L = 256$ bytes.

Two fields are dropped. Field 3, the video release date, is *empty on all 1682 lines* — measured, not assumed — and field 4, the IMDb URL, is the largest field and is used by nothing downstream. What remains is `id | title | date | 19 genre flags`, whose longest instance is 117 bytes of content, leaving 139 bytes of headroom inside L. Every record is checked against that budget at load time, so a future dataset with a longer title fails loudly at startup rather than truncating silently at answer time.

Fields are carried with explicit one-byte length prefixes rather than a delimiter. Nine titles contain Latin-1 bytes and the file does not decode as UTF-8 at all, so it is read as bytes and never re-encoded; with arbitrary bytes in the payload, any delimiter choice risks colliding with content, whereas a length prefix cannot. Three records break a naive parser and each is handled explicitly: id 267 is `unknown` with an empty date and empty URL, id 1242 is the width driver, and one release date is `4-Feb-1971` rather than the `DD-Mon-YYYY` that every other date follows — which never mattered, because dates are carried as opaque text and never parsed into a structured type.

The 1682 items pad to a domain of 2048, so db = 11. Padding records are all-zero. This leaks nothing precisely because record width is fixed regardless of content: a fetch of a padding slot is indistinguishable from a fetch of a real one.

4.2 The private read

The client wants record α without either server learning α. It generates a DPF key pair for the point function that is 1 at α, sends one key to each of P_0 and P_1, and each server expands its key over the whole domain and takes an inner product against the catalogue,

$$\mathrm{ans}_c[w] \;=\; \sum_{j=0}^{2^{\mathrm{db}}-1} \mathrm{EvalFull}(k_c)[j] \cdot D[j][w], \qquad c \in \{0,1\},$$

over the $W = L/(b/8)$ ring words of a record ($W = 32$ at $b = 64$). Because $\mathrm{EvalFull}(k_0)[j] - \mathrm{EvalFull}(k_1)[j]$ is 1 at $j = \alpha$ and zero elsewhere, the *difference* of the two answers is exactly $D[\alpha]$. Note the difference rather than the sum: this project omits the $(-1)^c$ factor, per Section 3.2.

Two properties carry the obliviousness claim, and they are worth separating because the second is the stronger one:

1. The server touches *every* record, in index order, with no data-dependent branch. The loop bound is the domain size and nothing inside the loop depends on α.
2. Each expanded coefficient is a *share*, so it is pseudorandom at every index rather than zero at all but one. There is no data pattern for a compiler to exploit or a cache-timing observer to see, even in principle. This is strictly more than "the code contains no branch".

The system has three servers but the DPF is $(2, 2)$. We fix P_0 and P_1 as the key holders; all three hold the catalogue, which is public data, so replication costs nothing in privacy.

What it costs. At db = 11 a query is a single **227-byte** key per server against a 2048-entry domain (Table 2), against 16 KB for the naive one-hot query the same read would otherwise need: a 72× reduction in upload. The server's work is one EvalFull plus $2^{11} \times 32$ ring multiply-accumulates.

Verification. Reads are checked byte-exactly against a cleartext lookup: 60 random indices at $b = 64$ and 20 at $b = 128$, plus both domain endpoints, padding slots, and rejection of a key whose domain width does not match the server's.

One test in this group was wrong, and establishing that took more work than accepting it would have. A natural check — "no word of one server's share should equal the plaintext" — fails, with 18 of 32 words matching. That is not a leak. Records pad to 256 bytes while content never exceeds 117, and the trailing genre bytes are usually zero, so the upper words are identically zero across the *entire* catalogue; any linear combination of zeros is zero, for every α equally. It is public structure, not a signal. The test now derives from the data which words actually vary — 14 of 32 at $b = 64$ and 7 of 16 at $b = 128$, which agree at 112 bytes and so cross-check each other — and asserts only on those.

4.3 Scoring under secret sharing

User embeddings **A** are secret; item embeddings **B** are public, which is NUDGE's design and is what makes its power iteration cheap. The user's row is shared 2-of-3, and because **B** is public, $\mathbf{s} = \mathbf{a}^{(i)} \cdot \mathbf{B}$ is a *linear map applied to a shared vector by a public matrix*. Each server computes its own share of all n scores entirely locally: no communication, no correlated randomness, no multiplication protocol, no rounds. This is the single largest saving in the serving path and it comes directly from **B** being public.

Scores stay at $2t$ scale. Both factors are encoded at t fractional bits, so the product carries $2t$. Truncating back to t requires Trunc_t, which is interactive and belongs to the training phase, so serving decodes at $2t$ instead. This is recorded rather than left implicit because getting it wrong scales every score by 2^{20}, which is *monotone* and therefore completely invisible in a ranking.

The seen-item mask. The ideal functionality sets already-rated items to $-\infty$, which a finite ring cannot represent. We use an additive shared vector carrying $-(2^{55})$ at seen positions and zero elsewhere; servers add it to their score shares, which is again local. Observed encoded scores peak near 9.7×10^{12} and the ring floor is near -9.2×10^{18}, so the sentinel sits about 3700× below any real score and about 256× above the floor. Both margins are asserted by a test rather than argued in prose. The comparison in the top-k selection is *signed*: comparing these two's-complement values as unsigned would sort masked items *first*, and a smoke test inspecting only the head of the list would not notice.

We state one thing plainly rather than overclaim it: in this phase the user reconstructs the score vector anyway, so masking server-side is *equivalent* to masking client-side. It is implemented server-side because that is what the ideal functionality specifies and what the composed system will need once the servers do more than a linear map — not because this phase requires it.

4.4 The float-to-ring boundary

Exactly one program converts a float into a ring element: `model/export.py`. Everything downstream trusts it, so it is tested as a contract rather than assumed. A committed file of 25 `<double> <int64>` pairs — zero, both signs, the quantum and half the quantum, the observed extremes of **A**, **B** and the scores, and the two's-complement edge — is read by the C++ test suite, so the boundary is checked on a fresh clone with no export run.

It found two bugs immediately, both of which would have been hard to find later.

Rounding mode. `numpy.rint` and Python's `round` use banker's rounding, half to even, so rint(0.5) = 0; C++ `std::round` rounds half away from zero, so round(0.5) = 1. They disagree at every exact half-quantum. The contract test caught this on its first run. Python was changed to match C++, since C++ is what ships.

Quantise-then-multiply is not multiply-then-quantise. The reference scores were originally computed by encoding the float product; the protocol multiplies encoded factors, which rounds d times and then accumulates exactly. The two disagreed on **1650 of 1682 scores**, by up to 2.1×10^7 out of 4.3×10^{12}. That is about 5×10^{-6} relative and irrelevant to the ranking, which is exactly why it matters: any tolerance-based comparison would have accepted it, and the oracle would have been quietly wrong for every later exact test. The reference now computes what the protocol computes, in arbitrary-precision integers.

Serving is therefore checked against the oracle by **exact** equality, not by tolerance. A linear map over a ring has no tolerance to spend, and a tolerance here would hide precisely the endianness, sign-extension and scale bugs the boundary is prone to. All 1682 scores match bit-for-bit.

Ring width, measured. An open question in the design was whether $b = 64$ suffices at MovieLens scale or whether $b = 128$ is needed. Measured on the actual factorisation: **A** peaks at 61.4, **B** at 0.219 (every row has unit L_2 norm by construction), and at $t = 20$ the worst-case $d = 16$ accumulation is **47.8 bits of the 63 available** — roughly a 32,000× margin. Quantisation costs $\max|S_q - S| = 5.3 \times 10^{-5}$, far below the gaps between top-20 scores, so the ranking is preserved. So $b = 64$ is sufficient at this scale, and the implementation remains templated on the ring so the $b = 128$ arm is tested alongside it at every step.

4.5 End to end

`demo -user 42 -k 10` runs the whole path: the embedding is split across three servers, each scores the catalogue locally, the user reconstructs and selects the top ten, and each of the ten records is then fetched by a two-server DPF-PIR read. It prints ten real film titles, and it checks itself twice: every fetched record is compared byte-for-byte against a cleartext lookup, and the ranking is compared position-for-position against the independently computed Python reference. Both agree.

For user 42, masking the 112 items already rated in the training split, the top ten are *The Princess Bride*, *Indiana Jones and the Last Crusade*, *Toy Story*, *Jerry Maguire*, *Apt Pupil*, *Good Will Hunting*, *Back to the Future*, *The Rock*, *Mission: Impossible* and *Aladdin* — fetched at a cost of one 227-byte key per server per title, with neither server able to say which titles they were.

5 What an observer learns without the private read

The previous section built a private read. This one measures what happens without it, and it is the strongest result we have: it is the quantitative answer to why the delivery layer must be private at all, and neither reference paper reports it, because neither implements both halves of the system.

5.1 Why a fetch is not one bit

NUDGE publishes the item embedding matrix **B** in the clear. That is the design and not an oversight: each converged row of **B** is opened before the next is computed, which is what makes SETORTHOGONAL a Gram–Schmidt against *public* vectors and keeps the whole power-iteration approach affordable under secret sharing.

The consequence is the one this section measures. **B** is a complete latent-factor model of the catalogue, known to everyone. So an adversary who observes *which* records a user fetched does not learn j isolated facts; each fetch is a constraint on the user's private embedding $\mathbf{a}^{(i)}$, a projection onto a basis the adversary already holds. The question is how fast those constraints collapse.

5.2 The experiment

The adversary is given **B** and the *indices* of the j records the user fetched. It is given neither the scores nor **A**. It computes $\hat{a}$, the centroid of the fetched items' unit embeddings, and we report $\cos(\hat{a}, \mathbf{a}^{(i)})$ together with the top-20 overlap between the ranking $\hat{a}$ induces and the user's true ranking. All 943 users, at $d = 16$, $\ell = 10$, against the same factorisation the demo serves.

The control is the load-bearing part of the design. A random-direction control would flatter any attack of this shape. Popular items sit near the top of nearly everyone's ranking, so an estimator can score well having learned only what is *public*. The honest control therefore runs the identical estimator on the j globally most-popular items, ignoring the user entirely, and the attack may claim only what it achieves *above* that line. The random control is retained to show where the floor actually is.

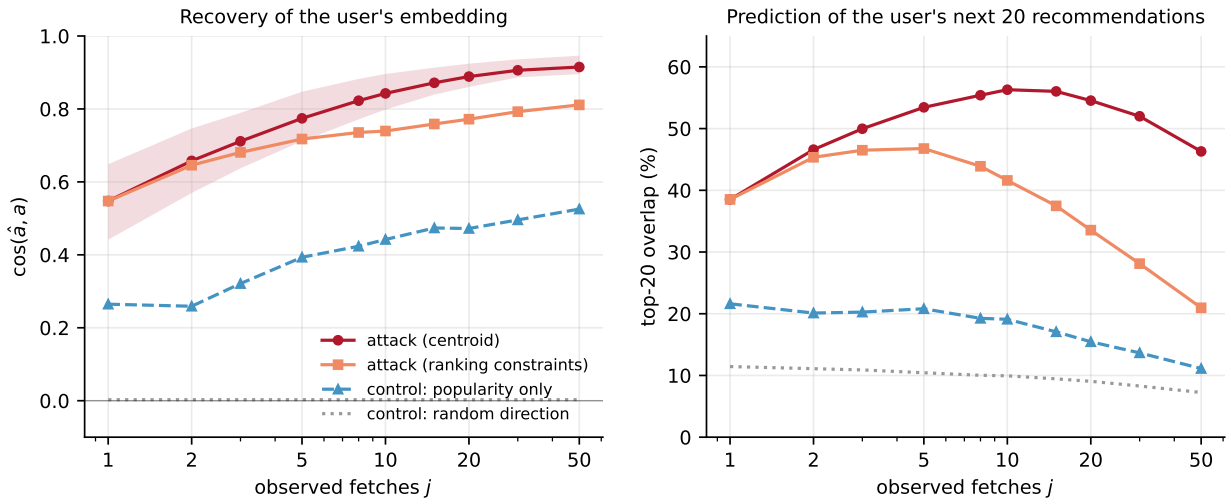

Figure 3: Reconstruction of a user's private embedding from observed fetches alone, over all 943 users. **Left:** cosine against the true embedding, with the interquartile band on the attack. **Right:** the share of the user's *next* twenty recommendations the adversary can name. Both panels share the same controls. The gap between the attack and the popularity control is the part that is genuinely about the individual rather than about what is popular, and it is stable at +0.39 to +0.42 across every j. Regenerated by `mingw32-make figures` from `bench/results/leakage_attack.jsonl`.

5.3 Result

A single observed fetch already gives cos = 0.55**, twice the popularity control. Ten give** 0.84**, and are enough to predict 56% of that user's next twenty recommendations.** The random control sits at 0.003.

Two caveats we raise ourselves rather than wait to be asked.

Table 3: Recovery against the number of observed fetches. The overlap column excludes the j items already seen, so it measures prediction of the user's *next* recommendations rather than repetition of the ones handed to the adversary.

j	$\cos(\hat{a}, a)$	popularity control	next-20 overlap
1	**0.55**	0.26	38%
2	0.66	0.26	47%
5	0.77	0.39	53%
10	**0.84**	0.44	**56%**
20	0.89	0.47	55%
50	0.92	0.53	46%

First, *an individual user's cosine is not evidence*. The random control's interquartile spread is roughly ± 0.18, near $1/\sqrt{d}$ at $d = 16$, so one user scoring 0.2 is inside the noise of guessing. Only population means, and the margin over the popularity control, carry a claim.

Second, the overlap column is *non-monotone*: it peaks near $j = 10$ and falls by $j = 50$. That is a property of the metric, not a degradation of the attack. The j observed items are excluded from the candidate pool, so as j grows the easy head of the ranking is progressively removed and what remains is a harder tail. The cosine rises monotonically throughout, which is the check that the estimate itself keeps improving.

An honest negative result. We also implemented a second, stronger estimator, which uses the fact that the fetched set is the *top* of the score vector and so every fetched item outranks every unfetched one — strictly more information than the centroid uses. It is *worse* at every $j \geq 2$ (0.81 against 0.92 at $j = 50$). We report it rather than quietly dropping it, because it sharpens the conclusion rather than weakening it: the cheap, obvious attack is already sufficient, so an adversary here needs no sophistication.

5.4 What it costs to close the gap

Figure 4 prices it. Against B5, downloading the entire catalogue and filtering at home — which is trivially private and is the first thing a reader proposes, since 430 KB is not obviously unreasonable — DPF-PIR sends **446× fewer bytes** counting both servers.

Against B1, a cleartext lookup with no privacy at all, it costs about **50,000× more server time**. And read plainly, **DPF-PIR is the slowest of the three in server CPU**, about 5× slower than B5's local copy. Its advantage is entirely in communication.

Setting the extra computation against the saved bytes, the private read is the better choice on any link slower than **8.0 Gbit/s** — which is to say on every real network, but we would rather state the crossover than imply there is not one. Absolute timings on this machine vary up to 4× between back-to-back runs, so these are ratios taken within a single run, as Section 6 explains.

Together the two halves of this section make the argument the composition exists to make: without a private read, ten observed fetches recover the user; with one, the cost of hiding them is a few hundred bytes and a fraction of a millisecond. NUDGE leaves this to "other means". This is what those means cost.

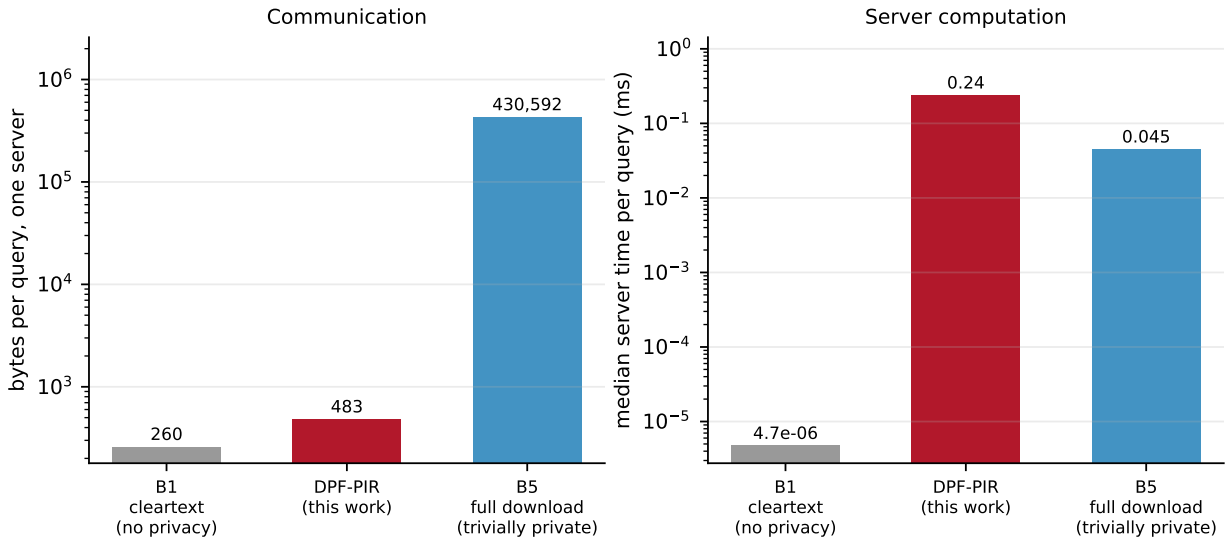

Figure 4: The private read against the two baselines a reader would otherwise propose, at the ML-100K operating point (1682 items, 256 B records, db = 11). Both panels must be read together; showing only the left one would be advocacy rather than evaluation.

6 What the primitive costs

Correctness settles whether the construction is right. This section settles whether it is affordable, which is the question that decides if the delivery layer is buildable at all.

6.1 How the numbers are produced

Every measurement is emitted as one JSON object per line, carrying the fields fixed by the design draft: the commit hash, the host, the network profile, the parameter tuple, the pipeline stage and phase, wall time, and bytes sent. The commit hash, host and timestamp are filled by the emitter rather than by the caller, so a row cannot exist without provenance. The phase is validated against the permitted set on the way out, because a misspelled phase produces a row that a plotting script silently drops, and that is discovered days later.

Two conventions are worth stating. Wall time is recorded *per operation*, with the batch size carried alongside so batch time is recoverable. And one row is written per repetition, never an aggregate, because the raw file is meant to be raw and aggregation inside the producer would hide the distribution.

The benchmark writes to standard output and its human-readable table to standard error, with the redirection living in the build system. The binary therefore has no file handling and no assumption about the working directory.

Guarding against the compiler optimising away the work being timed is partly structural, since the primitive lives in a separate translation unit with link-time optimisation switched off, so its bodies are not visible to be elided. That would stop being true the moment anyone enabled it, so every result is additionally accumulated into a live value that is forced to memory and printed as a checksum. Query indices are precomputed outside the timed region, because generating them inside it would time the generator, and a strictly increasing sweep would give the branch predictor an unrealistically easy task on the tree path. The whole-domain output buffer is allocated once,

outside every loop, since a freshly zeroed eight megabyte buffer per call would measure the page fault handler rather than the primitive.

6.2 Measured cost

Table 4: Median of five repetitions, 64-bit ring unless stated, on the development laptop with no network involved. Read the ratios rather than the absolute figures, for the reason given below.

domain bits	Gen (μs)	Eval (μs)	EvalFull (μs)	EvalFull, $b = 128$ (μs)
8	0.40	0.27	18.4	52.4
10	1.40	0.31	81.9	199.1
11	1.76	0.44	281.0	440.8
12	1.97	0.47	379.9	889.5
14	1.74	0.43	1352.7	4483.8
16	1.93	0.53	5536.0	14343.0

Three things follow.

The delivery layer is affordable at the scale this project targets. At eleven domain bits, which is the padded MovieLens-100K catalogue, one server answering one query costs a single whole-domain evaluation, measured at roughly 0.28 ms. Key generation on the client is under two microseconds and the key itself is 227 bytes. Nothing here is close to being the bottleneck.

Whole-domain evaluation earns its place. Evaluating the entire domain through repeated single-point evaluation is between three and four times slower than the depth-first whole-domain pass across the range measured, because the tree walk is shared rather than repeated from the root for every index. That ratio is the entire justification for the second entry point existing.

Key generation is not dominated by drawing randomness. A separate measurement of a thirty-two byte draw from the operating system generator gives about 0.07 μs, so the two draws a key needs account for well under a tenth of generation time even at the smallest domain. This was worth checking rather than assuming, because if it had gone the other way the sensible optimisation would have been buffering randomness rather than touching the tree, and that is not where the cost is.

6.3 A caveat about the absolute figures

These numbers were taken on a laptop. Running the benchmark repeatedly in quick succession produced absolute timings differing by as much as a factor of four between runs, while the spread across repetitions *within* a single run stayed between 1.1 and 1.7. That pattern is consistent with thermal throttling rather than with measurement noise.

The consequence is stated plainly rather than smoothed over. Ratios between operations measured in the same run are trustworthy. Absolute figures should be read as orders of magnitude, and the conclusions above are drawn only from ratios and from margins wide enough that a factor of four does not change them. Every repetition is retained in the raw file with its own timestamp, so the drift is visible to anyone who looks rather than being averaged away. Obtaining figures that would survive being quoted precisely needs a machine that is not thermally constrained, and that is deferred rather than faked.

References